

AI Copilot Usage Log (Curated)

Session Metadata

- Unit: NIT2112 Object Oriented Programming
- Project: OzZoo (`ozzoo\_gui`)
- Session date: 2026-04-10
- Focus: UI fixes, gameplay expansion, debugging, balancing, and documentation

This is a curated log (not a raw transcript). It captures high-impact prompt cycles, representative AI outputs, and why each prompt was used.

Key Prompt 1 - UI Bug Fix (Shop Purchase Dialog)

Prompt

`On the shop frame I can not purchase animals still, the pop-up dialog does not have a purchase button`

AI Output Snippet

```
dialog = tk.Toplevel(self._app)
dialog.title(f"Purchase {info['display_name']}")
dialog.geometry("450x500")
```

Analysis

I used this prompt to isolate a usability bug. The AI identified dialog sizing as the root cause and increased the window height so purchase controls render reliably.

Key Prompt 2 - Wrong Habitat Demonstration and Safety Constraint

Prompt

`Now in the src code there is mention of being able to put the animals in the wrong cages ...`

`I would like to remove the visual indicators and not allow the purchase ... replace yes/no with okay`

AI Output Snippet

```
if not is_compatible:
    from tkinter import messagebox
    messagebox.showwarning(
        "Wrong Habitat Warning!",
        f"{name} ({info['display_name']}) cannot be placed in this enclosure!\n\n"
        f"Please select a compatible enclosure."
    )
    return
```

Analysis

I used this prompt pair to first expose mismatch behaviour, then tighten rules so incompatibility is educational but non-destructive. The result keeps warning feedback while preventing invalid purchases.

Key Prompt 3 - Gameplay Expansion (Mission Keeper + Dynamic Start)

Prompt

`... random keeper ... random mission ... random map ... add enclosures ... highlight selected tile`

AI Output Snippet

```
mission = generate_starter_mission(difficulty="easy")
zoo.start_mission(mission)

mission_keeper = generate_random_keeper("keeper_mission")
zoo._mission_keeper = mission_keeper
zoo.hire_keeper(mission_keeper, signing_bonus=0)

required_habitat = get_mission_animal_habitat(mission)
zoo.add_enclosure(create_enclosure(required_habitat, enc_id, name=enc_name))
```

Analysis

I used this prompt to shift the game from static setup to replayable progression. The AI output introduced a keeper-driven mission loop and mission-dependent starting map state.

Key Prompt 4 - Debugging New-Game Crash

Prompt

`Error. Failed to create new game: Unknown enclosure type: outback\_burrow...`

AI Output Snippet

```
enclosure_name_map = {
    "eucalyptus_grove": "Koala Corner",
    "outback_savanna": "Outback Plains",
    "billabong": "Crocodile Creek",
    "rainforest_aviary": "Bird Paradise",
    "reptile_house": "Reptile Realm"
}
```

Analysis

I used this prompt as a targeted regression fix. The AI aligned mission/startup mappings with valid enclosure types to remove the runtime failure path.

Key Prompt 5 - Reviews, Ratings, and Reputation

Prompt

`Now reviews from our visitors are not affecting the game state... add rating like '4 out of 5 stars' ... add Reputation next to Welfare`

AI Output Snippets

```
def get_star_display(self) -> str:
    stars = "■" * self.rating
    return f"{stars} {self.rating} out of 5 stars"

daily_reviews = generate_reviews(
    self._animals,
    self._day,
    visitor_count=visitor_count_for_reviews
)
self._reviews.extend(daily_reviews)
report["reviews"] = daily_reviews
self._reputation = calculate_reputation_from_reviews(self._reviews)

("■ Welfare", f"{report['avg_welfare']:.0f}%", self._get_welfare_color(report['avg_welfare'])),
("■ Reputation", f"{self._zoo.reputation:.0f}/100", self._get_reputation_color(self._zoo.reputation)),
```

Analysis

I used this prompt to connect sentiment feedback to game state. The AI added a full review model, daily generation tied to welfare, star-based review text in the day popup, and dashboard reputation visibility.

Key Prompt 6 - Economy Balance Bug (Map Enclosure Costs)

Prompt

`Bugs: Buying enclosures in the Map frame are not supposed to be free.`

AI Output Snippets

```
PLACEMENT_COSTS = {
    "eucalyptus_grove": 5000,
    "outback_savanna": 6000,
    "billabong": 7000,
    "rainforest_aviary": 5500,
    "reptile_house": 6500,
}

cost = PLACEMENT_COSTS.get(enclosure_type, 0)
if cost > 0 and self._zoo.budget < cost:
    self._app.show_warning("Insufficient Funds", ...)
    return

self._zoo._budget -= cost
self._zoo._total_expenses += cost
```

Analysis

I used this prompt to restore economic integrity. The AI changed enclosure placement from free to budget-constrained with explicit insufficiency feedback.

Documentation Prompt (This Deliverable)

Prompt

`Create an AI Copilot Usage Log (PDF format) from this session ... curated log ...`

AI Output Snippet

Curated markdown source compiled into PDF with key prompts, code snippets, and rationale for each intervention.

Analysis

I used this prompt to transform technical work into assessment evidence. The output emphasises decision quality and traceability, not just final code.

Session Deliverables

1. `AI\_Copilot\_Usage\_Log\_Session\_2026-04-10.md` (curated source)
2. `AI\_Copilot\_Usage\_Log\_Session\_2026-04-10\_Curated.pdf` (final PDF)